PROGRAMACIÓN III — IA 3.2

Resumen completo orientado al parcial

Unidades 1 – 4 | TUIA – FCEIA UNR

⚑ **ÍNDICE RÁPIDO:** U1: Definición formal de un problema de búsqueda

U2: Estrategias No Informadas (BFS · DFS · UCS · IDS)

U3: Búsqueda Informada (GBFS · A*) · Búsqueda Local · Juegos · CSP

UNIDAD 1: Definición formal de un problema de búsqueda

1.1 Las 5 componentes de un problema

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Se pide en TODOS los parciales: enumerar las 5 componentes de un problema de búsqueda.

#	Componente	Descripción / Ejemplo
1	Estado inicial	Configuración de partida del agente.
2	Función de acciones	Qué puede hacer en cada estado → ACCIONES(s).
3	Modelo de transición	Resultado de aplicar una acción → RESULTADO(s, a).
4	Test objetivo	Verifica si el estado actual es solución.
5	Función de costo de camino	Valor numérico asignado a la secuencia de acciones.

1.2 Nodo vs. Estado

⚑ **Estado:** Configuración del mundo (ej: posición de las fichas en el 8-puzzle).

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Los atributos de la clase Nodo son los 5 siguientes — pregunta fija en parciales.

Atributo	Tipo / Descripción	
estado	El estado representado por este nodo.	
acción	La acción aplicada para llegar aquí.	
padre	Nodo anterior en el camino.	

costo de camino (g)	Costo acumulado desde el inicio.	
profundidad	Nivel en el árbol de búsqueda.	

1.3 Frontera y conjunto de expandidos

- **Frontera:** nodos generados pero no expandidos todavía.
- **Expandidos (lista cerrada):** estados ya explorados. Solo existe en la búsqueda de grafo.

Variante	¿Usa lista de expandidos?
Búsqueda de árbol	NO — puede repetir estados → bucles infinitos.
Búsqueda de grafo	SÍ — evita redundancias. Recomendada para espacios con caminos redundantes.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** ¿Cuál se recomienda con caminos redundantes? → Búsqueda de GRAFO.

1.4 Criterios de evaluación

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Los 4 criterios se preguntan en múltiples parciales. Memorizarlos con sus definiciones.

Criterio	Pregunta que responde
Completitud	¿El algoritmo siempre encuentra solución si existe?
Optimalidad	¿Encuentra siempre la solución de menor costo?
Complejidad temporal	¿Cuántos nodos genera / cuánto tiempo demora?
Complejidad espacial	¿Cuántos nodos almacena en memoria?

UNIDAD 2: Estrategias de búsqueda no informada

2.1 Búsqueda en Anchura (BFS)

- **Estructura de frontera:** Cola (FIFO).
- **Completo:** Sí (si b es finito).
- **Óptimo:** Sí, para costos unitarios.
- **Memoria (árbol):** O(b^(d+1)) — almacena TODOS los nodos del nivel d+1.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** ¿Cuántos nodos almacena BFS (árbol) en peor caso? → b^(d+1). Opción correcta siempre.

Orden BFS: Nivel por nivel. Si A tiene hijos B y C, y B tiene hijos D y E, el orden es A → B → C → D → E → ...

⚑ **¡CUIDADO!:** BFS puede procesar los hijos en distinto orden: A,B,C,G,F,E y A,B,C,E son ambos válidos si el árbol lo permite. Hay que mirar el árbol dado.

2.2 Búsqueda en Profundidad (DFS)

- **Estructura de frontera:** Pila (LIFO).
- **Completo:** No en espacios infinitos; sí en grafos finitos.
- **Óptimo:** No.
- **Memoria:** O(b·m) — muy eficiente (solo almacena la rama actual).

2.3 Búsqueda de Costo Uniforme (UCS)

- **Estructura de frontera:** Cola de prioridad (ordena por g(n) creciente).
- **Completo:** Sí.
- **Óptimo:** Sí, para CUALQUIER función de costo con valores positivos.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Ventaja de UCS sobre BFS: es óptimo para cualquier costo positivo (no solo costos unitarios).

`UCS: expande el nodo n que minimiza g(n)`

2.4 Búsqueda de Profundidad Iterativa (IDS)

Realiza búsquedas DFS con límite de profundidad creciente (ℓ = 0, 1, 2, …) hasta encontrar solución.

- **Completo:** Sí.
- **Óptimo:** Sí (como BFS).
- **Memoria:** O(b·d) — como DFS.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Si IDS llega al límite ℓ y no encuentra solución → aumenta en 1 el límite y sigue buscando (o termina si hay criterio de parada).

2.5 Tabla comparativa de algoritmos no informados

Algoritmo	Frontera (TAD)	Óptimo / Completo
BFS	Cola (FIFO)	✓ / ✓
DFS	Pila (LIFO)	✗ / ✗
UCS	Cola de prioridad	✓ / ✓
IDS	Pila (LIFO) + límite	✓ / ✓

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Pregunta fija: ¿qué TAD usar para BFS / DFS / UCS? → Cola / Pila / Cola de prioridad.

UNIDAD 3: Estrategias de búsqueda informada

3.1 Función heurística h(n)

⚑ **Heurística h(n):** Estimación del costo desde el estado del nodo n hasta el objetivo más cercano.

3.2 Búsqueda Voraz (Greedy Best-First Search — GBFS)

$$f(n) = h(n)$$

- **Estrategia:** expande el nodo que parece más cercano al objetivo.
- **Rápida:** Sí. Óptima y completa: NO garantizadas.

3.3 Algoritmo A*

$$f(n) = g(n) + h(n)$$

- g(n): costo real desde el inicio hasta n.
- h(n): estimación del costo desde n hasta el objetivo.
- f(n): estimación del costo total del camino que pasa por n.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Diferencia clave: GBFS minimiza h(n), A* minimiza g(n)+h(n), UCS minimiza g(n).

3.4 Condiciones de optimalidad de A*

Variante de A*	Propiedad requerida de h
A* de ÁRBOL	Admisible: h(n) ≤ h*(n) (nunca sobreestima el costo real)
A* de GRAFO	Consistente (monótona): h(n) ≤ costo(n,a,n') + h(n')

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Para garantizar solución óptima en búsqueda de GRAFO → A* con heurística CONSISTENTE.

⚑ **CLAVE:** Toda heurística consistente es también admisible, pero no viceversa.

3.5 Construcción de heurísticas

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Se preguntan las 3 estrategias para construir heurísticas informadas.

1. **Problemas relajados:** eliminar restricciones del problema. Ej: distancia Manhattan en 8-puzzle (ficha puede moverse aunque haya obstáculos).
2. **Subproblemas / bases de datos de patrones:** resolver una versión parcial del problema y tabular los costos.
3. **Heurísticas compuestas:** tomar el máximo de varias heurísticas admisibles → sigue siendo admisible y es más informativa.

3.6 Distancia Manhattan — Cálculo

Para cada ficha numérica, calcular |fila_actual − fila_objetivo| + |col_actual − col_objetivo|. Sumar todos.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Ejemplo del parcial (estado: 1-2-5 / 3-6-4 / _-7-8 vs objetivo _-1-2 / 3-4-5 / 6-7-8): h = 6.

BÚSQUEDA LOCAL (Unidad 3 ampliada)

4.1 Conceptos generales

La búsqueda local NO guarda el camino ni la frontera. Solo mantiene el estado ACTUAL y lo modifica buscando mejorar la función objetivo.

- **Ventaja:** muy bajo consumo de memoria.
- **Uso típico:** problemas de optimización como n-reinas, TSP, etc.

4.2 Paisaje del espacio de estados

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Se piden los nombres de los 4 tipos de puntos en el paisaje de estados.

Tipo de punto	Descripción
Máximo global	El mejor estado posible de todo el espacio.
Máximo local	Mejor que todos sus vecinos, pero no el mejor global.
Máximo local plano (plateau)	Zona donde todos los vecinos tienen el mismo valor.
Terraza	Zona plana con un borde que desciende (no todos los vecinos son iguales).

4.3 Ascensión de colinas (Hill Climbing)

Algoritmo greedy de búsqueda local. Siempre se mueve al estado sucesor con MAYOR valor objetivo.

⚑ **Pseudocódigo clave (líneas 6–7):** if problema.fobj(sucesor) <= problema.fobj(actual) then → return actual

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Si se pide minimizar (ej. pares de reinas atacadas), la condición de parada cambia: se detiene si el sucesor tiene mayor o igual valor.

Característica topográfica	¿Puede escapar el Hill Climbing estándar?

Máximo local	NO — queda atrapado.
Máximo local plano / plateau	Con pasos laterales: puede escapar. Sin pasos laterales: NO.
Terraza	Con pasos laterales: SÍ (puede escapar de la terraza).

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** ¿De qué escapa el movimiento lateral?
→ De la TERRAZA (y del máximo local plano en algunos casos).

⚑ **¡CUIDADO!:** Hill Climbing con pasos laterales puede seguir caminos CÍCLICOS. Sin pasos laterales, no.

4.4 Búsqueda Tabú

Variante de Hill Climbing que mantiene una lista tabú de estados/acciones/propiedades recientemente visitados, para evitar repetirlos.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Elementos que suele almacenar la lista tabú: (1) Estados · (2) Acciones · (3) Propiedades.

⚑ **¡CUIDADO!:** La búsqueda tabú SÍ puede seguir caminos cíclicos (si la lista tiene tamaño limitado).

4.5 Recocido Simulado (Simulated Annealing)

Permite movimientos "hacia abajo" (peores estados) con una probabilidad que disminuye con el tiempo.

```
P(aceptar s') = e^(Δf / T(i))
```

Donde **Δf = f(s') − f(s)** (diferencia de valor objetivo) y **T(i)** es la temperatura en la iteración i.

La temperatura se calcula con el esquema de enfriamiento: $T(i) = t_0 \cdot c^i$

⚑ **CÁLCULO DEL PARCIAL:** $t_0 = 10000$, $c = 0.95$, $i = 50$, $f(s) = 50$, $f(s') = 10$

$T(50) = 10000 \times 0.95^{50}$

$\Delta f = 10 - 50 = -40$

$P = e^{(-40 / T(50))} \approx 95\%$

4.6 Algoritmos Genéticos

Metaheurística poblacional inspirada en evolución biológica.

- **Individuo:** solución candidata (ej: una permutación de ciudades en TSP).
- **Fitness:** valor de la función objetivo.
- **Operadores:** selección, cruce (crossover), mutación.

⚑ **Representación ordinal (TSP):** Dado Orden = ⟨1,2,3,4,5⟩ y Tour = ⟨2,5,1,4,3⟩: para cada ciudad del tour, contar su posición relativa en el orden restante. Resultado: ⟨2,4,1,2,1⟩.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Ejercicio tipo: dado un Tour y un Orden, calcular la representación ordinal.

JUEGOS ENTRE ADVERSARIOS

5.1 Formulación

Juegos de dos jugadores (MAX y MIN), suma cero, información perfecta, deterministas.

Componente	Descripción
S_0	Estado inicial.
JUGADOR(s)	A quién le toca mover (MAX o MIN).
ACCIONES(s)	Movimientos legales.
RESULTADO(s,a)	Estado resultante.
TEST-TERMINAL(s)	¿Terminó el juego?
UTILIDAD(s,p)	Valor final para el jugador p.

5.2 Algoritmo Minimax

- **MAX**: elige la acción que maximiza la utilidad.
- **MIN**: elige la acción que minimiza la utilidad.

Propiedad	¿Aplica al Minimax clásico?
Recorre el árbol completo	SÍ
Garantiza estrategia óptima	SÍ
Tiempo excesivo en la práctica	SÍ
Requiere función de evaluación	NO (solo con corte anticipado)

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Minimax clásico SÍ garantiza óptimo. Minimax con función de evaluación y test de corte: NO (es una aproximación).

5.3 Poda Alfa-Beta

Optimización de Minimax que elimina ramas innecesarias sin cambiar el resultado final.

- **α (alfa):** mejor valor garantizado para MAX hasta ahora.
- **β (beta):** mejor valor garantizado para MIN hasta ahora.
- **Poda:** si $\alpha \geq \beta$, se corta la búsqueda en esa rama.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Minimax con poda alfa-beta SÍ garantiza estrategia óptima (igual que Minimax clásico).

5.4 Juegos de suma cero

```
UTILIDAD(S, MAX) + UTILIDAD(S, MIN) = constante
```

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Ej: UTIL(S1,MAX)=4, UTIL(S1,MIN)=2 → suma=6. Entonces UTIL(S2,MIN)=6−3=3.

5.5 Juegos estocásticos (Expectiminimax)

Se agregan nodos de AZAR entre los turnos de los jugadores.

```
VALOR(Nodo_Azar) = Σ P(resultado_r) × VALOR_Minimax(resultado_r)
```

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** El turno es: MAX → AZAR → MIN → (hoja). Calcular el valor esperado en cada nodo AZAR, luego aplicar minimax normalmente.

PROBLEMAS DE SATISFACCIÓN DE RESTRICCIONES (CSP)

6.1 Las 3 componentes de un CSP

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Pregunta fija: enumerar las 3 componentes de un CSP.

4. **Variables:** $X_1, X_2, \ldots, X_n$
5. **Dominios:** $D_1, D_2, \ldots, D_n$ (valores posibles para cada variable)
6. **Restricciones:** condiciones que deben satisfacerse entre variables.

6.2 Tipos de restricciones

- **Binaria:** involucra dos variables. Ej: $\langle(X_1,X_2), X_1 \neq X_2\rangle$
- **Global:** involucra múltiples variables. Ej: $\text{Alldiff}(X_1,X_2,X_3)$

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Para reemplazar 'todos distintos entre sí' → usar una única restricción: $\langle(X_1,X_2,X_3), \text{Alldiff}(X_1,X_2,X_3)\rangle$

6.3 Arco-Consistencia

⚑ **Arco-Consistencia:** Una variable X_i es arco-consistente con X_j si para cada valor en D_i existe al menos un valor en D_j que satisface la restricción entre ellas.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Ejemplo: $X_1 \in \{T,F\}$, $X_2 \in \{T,F\}$, restricción $X_1 \neq X_2$. Para $X_1=T$ existe $X_2=F$; para $X_1=F$ existe $X_2=T$ → SÍ es arco-consistente.

⚑ **CÁLCULO: Hacer X_1 arco-consistente con X_2:** Restricción: $2*X_1 = X_2$, $D_1=\{1,2,3,4\}$, $D_2=\{4,6\}$

$X_1=1$ → $2*1=2$, ¿$2 \in D_2$? NO → eliminar 1

$X_1=2$ → $2*2=4$, ¿$4 \in D_2$? SÍ → conservar

$X_1=3$ → $2*3=6$, ¿$6 \in D_2$? SÍ → conservar

$X_1=4$ → $2*4=8$, ¿$8 \in D_2$? NO → eliminar 4

Resultado: $D'_1 = \{2, 3\}$

6.4 Estrategias de propagación de restricciones

Estrategia	Descripción
Comprobación hacia adelante (FC)	Al asignar X_i, verifica arco-consistencia solo de las variables vecinas no asignadas respecto a X_i.
Mantenimiento de consistencia de arco (AC)	Propaga restricciones hacia toda la red tras cada asignación. Detecta MÁS inconsistencias que FC.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** ¿Cuál detecta más inconsistencias? → Mantenimiento de consistencia de arco (AC).

6.5 Algoritmos de resolución de CSP

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** Los 2 algoritmos más difundidos para CSP son: (1) Búsqueda vuelta atrás (backtracking) y (2) Heurística de mínimos conflictos.

Algoritmo	Características
Vuelta atrás (Backtracking)	Asigna variables una a una; si hay conflicto, retrocede. Se combina con FC o AC.
Mínimos conflictos	Búsqueda local: asignar valor que viole menos restricciones.

6.6 Estructura del grafo de restricciones

⚑ **Componentes conexas:** Si el grafo de restricciones tiene múltiples componentes conexas, cada una puede resolverse independientemente como subproblema.

⚑ **PREGUNTA FRECUENTE EN PARCIALES:** ¿Qué estructura permite descomponer en subproblemas? → Componentes conexas.

6.7 Restricciones — Formulación

⚑ **EJEMPLO DE PARCIAL:** Tarea A dura 10 min, tarea B dura 5 min.

B debe comenzar al menos 15 min después del FIN de A.

Restricción: ⟨(XA, XB), XA + 10 + 15 ≤ XB⟩

Es decir: XB ≥ XA + 25

HOJA DE REPASO RÁPIDO — Preguntas frecuentes del parcial

Pregunta tipo parcial	Respuesta correcta	Unidad
5 componentes de un problema	Est. inicial · Acciones · Modelo transición · Test objetivo · Costo camino	U1
5 atributos del Nodo	Estado · Acción · Padre · Costo camino · Profundidad	U1
4 criterios de evaluación	Completitud · Optimalidad · Tiempo · Memoria	U1
¿Árbol o grafo con caminos redundantes?	Búsqueda de GRAFO	U1
TAD para BFS / DFS / UCS	Cola / Pila / Cola de prioridad	U2
Memoria BFS árbol (peor caso)	b^(d+1)	U2
Ventaja UCS vs BFS	Óptimo para cualquier costo positivo (no solo unitarios)	U2
IDS: límite ℓ sin solución → ?	Aumenta ℓ en 1 y sigue buscando	U2
GBFS minimiza...	h(n)	U3
A* minimiza...	g(n) + h(n)	U3
A* árbol óptimo ↔ h es...	Admisible	U3
A* grafo óptimo ↔ h es...	Consistente	U3
3 formas de construir heurísticas	Relajados · Subproblemas / BD patrones · Compuestas	U3
¿Qué minimizar en 8-reinas?	Pares de reinas que se atacan	BL
Paisaje: tipos de punto	Máximo global · local · local plano · Terraza	BL
Movimiento lateral escapa de...	Terraza	BL
Búsqueda tabú almacena...	Estados · Acciones · Propiedades	BL
¿Tabú puede ser cíclica?	SÍ	BL
3 componentes de un CSP	Variables · Dominios · Restricciones	CSP
2 alg. principales CSP	Backtracking · Mínimos conflictos	CSP
Más inconsistencias detecta:	AC (Mantenimiento consistencia de arco)	CSP

Descomponer CSP en subproblemas →	Componentes conexas del grafo	CSP
Minimax clásico: ¿óptimo?	SÍ	Juegos
Minimax con eval + corte: ¿óptimo?	NO	Juegos
Poda alfa-beta: ¿óptima?	SÍ (misma que minimax)	Juegos
Juego suma cero: UTIL(S,MAX) +UTIL(S,MIN)=	Constante (= suma de las utilidades conocidas)	Juegos
TSP se clasifica como...	Problema de OPTIMIZACIÓN	General